

From Control to Offline Reinforcement Learning

Learning Better Decisions from Logged Data

Stephan Mandt

June 29, 2026

Predictions are not decisions

Supervised learning predicts	RL chooses
Truck arrival time	Which truck to unload next
Unloading duration	Which pit to assign
Grain quality	Which silo to route into
Train delay	How to schedule train loading

Business takeaway

Supervised learning predicts what happens next. Reinforcement learning chooses what should happen next.

Actions change the future



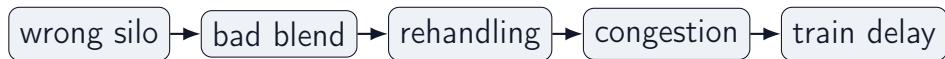
Choosing a truck, pit, route, or silo changes future queues, storage, blending options, and train schedules.

Product relevance

In RL, the model does not just observe the data distribution. It helps create the next data distribution.

Why imitation can fail

A behavior-cloned operator policy may be 99% accurate at each decision.



Business takeaway

A policy that is locally accurate can still be globally unsafe because it controls its own future inputs.

Why RL needs more than logs

simulator available



try novel actions
estimate con-
sequences

logs only



improve cautiously
stay near support

Business takeaway

RL is about policy improvement, not just action prediction.

The RL question

Given logged grain-terminal decisions and delayed outcomes, can we learn a better policy without risky online experimentation?

Logged data:

queue state, pit assignment, routing, silo choice, reward / KPI, next state

Product relevance

This is the bridge from product operations to the control and RL formalism.

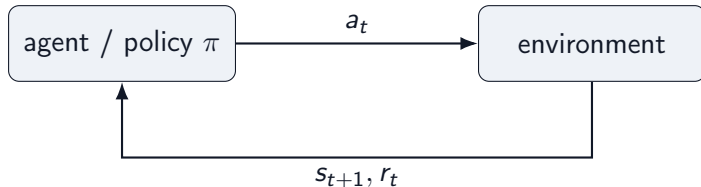
The minimal RL formalism

A Markov Decision Process:

$$(\mathcal{S}, \mathcal{A}, P, r, \gamma)$$

- ▶ $s \in \mathcal{S}$: state
- ▶ $a \in \mathcal{A}$: action
- ▶ $P(s' | s, a)$: transition
- ▶ $r(s, a)$: reward
- ▶ $\gamma \in [0, 1)$: discount

Agent-environment loop



Product relevance

In products, the “environment” may be customers, infrastructure, markets, robots, or agents.

Return and discounting

The return is cumulative future reward:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

A policy is good if its actions lead to high long-term return, not merely good immediate outcomes.

V, Q, and advantage

$V^\pi(s)$ = value of state s under policy π

$Q^\pi(s, a)$ = value of taking a in s , then following π

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

Business takeaway

Advantage asks: was this action better or worse than the policy's usual action in the same state?

Bellman thinking

$$Q(s, a) \approx r(s, a) + \gamma V(s')$$

The value of an action is immediate reward plus the value of where it takes us.

Product relevance

Bellman recursion is the RL version of cost-to-go recursion in optimal control.

Excursion: control and RL

Now that we have RL notation, classical control is easy to place:

controller $\approx$ policy, cost-to-go $\approx$ value function.

Both fields ask how to choose actions that improve future outcomes.

Product relevance

In the grain terminal, both a hand-designed controller and an RL policy are decision rules for routing, assignment, and loading.

Control $\leftrightarrow$ RL dictionary

Control theory	Reinforcement learning
State x_t	State s_t
Control u_t	Action a_t
Dynamics f	Transition $P(s' s, a)$
Cost $c(x, u)$	Negative reward $-r(s, a)$
Controller	Policy $\pi(a s)$
Cost-to-go $J(x)$	Value function $V(s)$
Dynamic programming / HJB	Bellman equations
MPC	Model-based RL / planning

Classical example: PID control

A PID controller chooses an action from tracking error:

$$u_t = K_P e_t + K_I \sum_{\tau \leq t} e_\tau + K_D (e_t - e_{t-1})$$

It is simple, interpretable, and often very strong when the control objective is local and well specified.

PID and RL: different strengths

Classical / PID control	Reinforcement learning
Simple and interpretable	Optimizes delayed objectives
Strong local feedback	Handles sequential tradeoffs
Easier to validate	Learns from data and simulation
Needs hand-designed structure	Can adapt to nonlinear operations
Usually optimizes a proxy	Can optimize the KPI directly

Business takeaway

Classical control is often the right starting point. RL becomes attractive when the best action depends on long-horizon operational consequences.

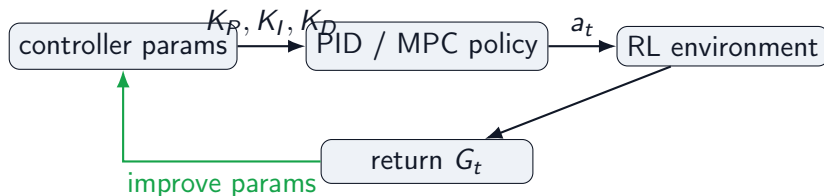
Why RL should improve long term

- ▶ It can optimize delayed outcomes, not only immediate tracking error.
- ▶ It can use simulators to evaluate actions not yet tried in production.
- ▶ It can learn terminal-specific interactions among queues, silos, conveyors, and trains.
- ▶ It can update as operating conditions and objectives change.

Product relevance

For grain loading, the best local move may not be the best move for throughput, quality, rehandling, and train delay.

The worlds can be combined



Business takeaway

A classical controller can be the policy class. RL can learn or tune its parameters from return.

Online RL loop



Online RL learns by interacting while training.

Three common RL families

Family	Basic idea
Value-based RL	Learn $Q(s, a)$, act greedily
Policy gradient	Directly improve $\pi(a s)$
Actor-critic	Learn both policy and value

Why online RL is hard in products

- ▶ Unsafe or costly exploration
- ▶ Customer-facing experiments
- ▶ Long feedback delays
- ▶ Sparse or noisy rewards
- ▶ Operational constraints and compliance

Product relevance

When exploration is expensive, risky, or product-constrained, logged data becomes the natural starting point.

Model-based RL: learn the dynamics, then plan

$$\hat{s}_{t+1} = \hat{f}_{\theta}(s_t, a_t)$$

$$a_{0:H}^* = \arg \max_{a_{0:H}} \sum_{t=0}^H \gamma^t \hat{r}(s_t, a_t)$$

Business takeaway

Model-based RL often looks like MPC with a learned dynamics model.

Risks: model bias, uncertainty, and compounding rollout error.

Offline RL setting

Fixed dataset:

$$\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$$

Behavior policy:

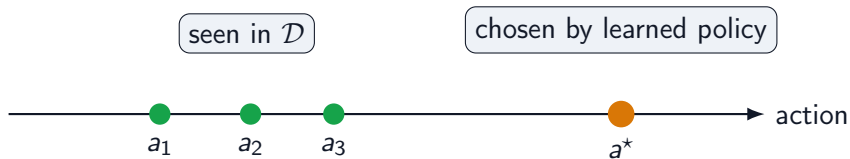
$$\mu(a \mid s)$$

Goal: learn $\pi(a \mid s)$ without online interaction during training.

Product relevance

Logged data source: terminal operator logs. Action space: truck selection, pit assignment, conveyor route, silo, and loading schedule.

Support mismatch



The policy may choose an unsupported action where the value estimate is unreliable.

Why naive Q-learning fails offline

Dangerous backup:

$$\max_a Q(s, a)$$

The max can select actions that were rarely or never observed.

Then the learned policy may exploit errors in Q outside the data distribution.

The offline RL thesis

Offline RL is mostly about preventing the policy from exploiting value errors outside the data distribution.

The rest of the talk builds a ladder:

$BC \rightarrow AWR \rightarrow IQL \rightarrow IDQL$

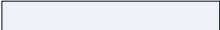
Behavior cloning

$$\mathcal{L}_{\text{BC}}(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi_{\theta}(a \mid s)]$$

Learn to imitate the actions in the dataset.

Benefits: stable, simple, scalable, no explicit value learning.

BC clones all actions equally

action A  low return

action B  medium return

action C  high return

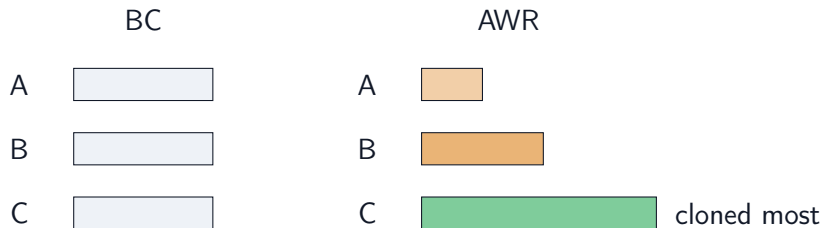
BC does not know which actions were good. It clones good, mediocre, and bad actions alike.

Advantage-weighted regression

$$\mathcal{L}_{\text{AWR}}(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{D}} [w(s, a) \log \pi_{\theta}(a \mid s)]$$

$$w(s, a) \propto \exp \left(\frac{A(s, a)}{\lambda} \right), \quad A(s, a) = Q(s, a) - V(s)$$

BC vs. AWR weighting



High-advantage actions are cloned more strongly; low-advantage actions are downweighted.

AWR is the first RL step

AWR is the smallest conceptual step from imitation learning to reinforcement learning.

It still looks like supervised learning, but the labels are now weighted by long-term value.

Product relevance

If logs contain good and bad decisions, AWR gives a way to imitate the better ones more.

IQL critic learning

Avoid explicit maximization over unseen actions:

$$\text{avoid: } \max_a Q(s, a)$$

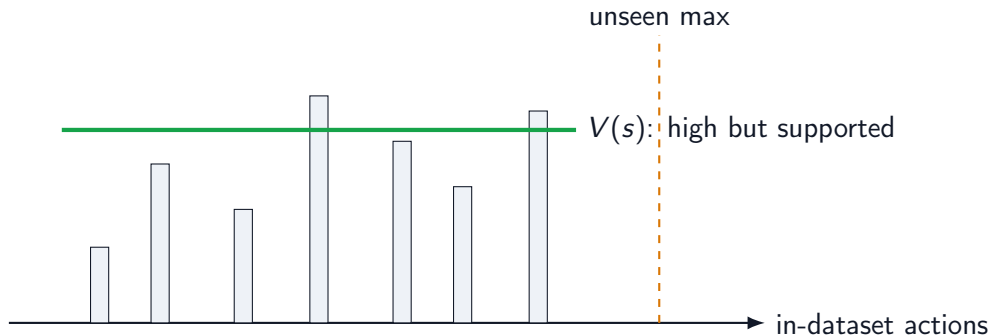
Dataset transition update:

$$Q(s, a) \leftarrow r + \gamma V(s')$$

Learn V as an upper expectile of in-dataset Q values:

$$\mathcal{L}_V = \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\left| \tau - 1_{Q(s,a) - V(s) < 0} \right| (Q(s, a) - V(s))^2 \right], \quad \tau > 0.5$$

In-sample value learning



IQ-L estimates a high value from in-dataset actions instead of maximizing over all actions.

IQL policy extraction

$$\mathcal{L}_{\pi} = -\mathbb{E}_{(s,a) \sim \mathcal{D}} \left[\exp \left(\frac{Q(s, a) - V(s)}{\beta} \right) \log \pi_{\theta}(a \mid s) \right]$$

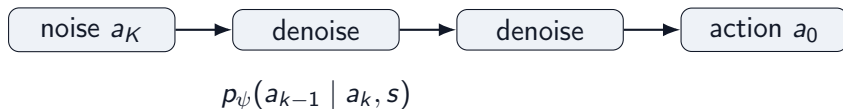
IQL uses advantage-weighted policy extraction after in-sample value learning.

IDQL: when the action distribution is multimodal



A simple Gaussian policy may average two good actions into a bad one.

Diffusion policies: generate actions by denoising



A diffusion policy represents $p_\psi(a \mid s)$ as an iterative conditional generation process [1].

Instead of predicting one mean action, it can sample diverse plausible actions.

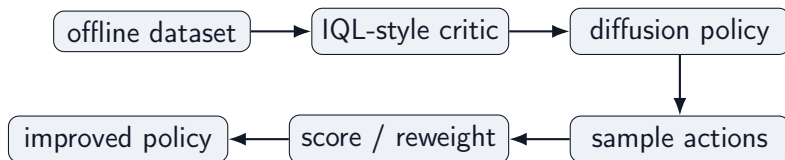
Why diffusion policies fit offline RL

- ▶ They can model multimodal action distributions.
- ▶ They stay anchored to logged behavior through behavior-model training.
- ▶ They expose multiple candidate actions for critic scoring.
- ▶ They are useful when action spaces are continuous, structured, or high-dimensional.

Business takeaway

Diffusion policies separate representation from improvement: sample plausible actions first, then use value estimates to prefer better ones.

IDQL pipeline



IDQL combines an IQL-style critic with diffusion-based policy extraction [2].

Business takeaway

IQL tells us which actions are valuable; IDQL gives us a richer way to represent and sample them.

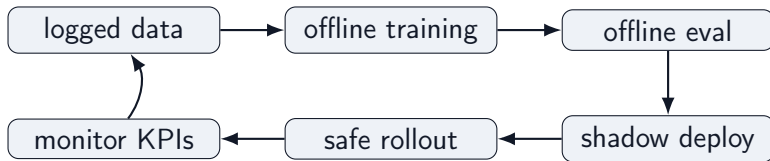
What matters in real offline RL systems?

1. **State quality:** is the decision approximately Markovian?
2. **Action support:** does $\mathcal{D}$ cover actions we want π to take?
3. **Reward design:** does reward reflect throughput, quality, delay, rehandling, and train-loading penalties?
4. **Counterfactual risk:** are we evaluating rare or unseen actions?
5. **Evaluation:** offline evaluation, simulation, shadow deployment, or cautious rollout?
6. **Policy constraints:** how far may π move from μ ?

Product relevance

Safe deployment strategy: shadow recommendations, operator approval, and constrained rollout.

Product architecture sketch



Closing thesis

RL is data-driven optimal control.

Offline RL is controlled policy improvement from logged data.

The ladder:

control $\rightarrow$ RL $\rightarrow$ offline RL $\rightarrow$ BC $\rightarrow$ AWR $\rightarrow$ IQL $\rightarrow$ IDQL

References

-  Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song.
Diffusion policy: Visuomotor policy learning via action diffusion.
arXiv preprint arXiv:2303.04137, 2023.
-  Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jingyun G. Chen, and Sergey Levine.
IDQL: Implicit q-learning as an actor-critic method with diffusion policies.
arXiv preprint arXiv:2304.10573, 2023.
-  Ilya Kostrikov, Ashvin Nair, and Sergey Levine.
Offline reinforcement learning with implicit q-learning.
arXiv preprint arXiv:2110.06169, 2021.
-  Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu.
Offline reinforcement learning: Tutorial, review, and perspectives on open problems.
arXiv preprint arXiv:2005.01643, 2020.
-  Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine.
Advantage-weighted regression: Simple and scalable off-policy reinforcement learning.
arXiv preprint arXiv:1910.00177, 2019.
-  Richard S. Sutton and Andrew G. Barto.
Reinforcement Learning: An Introduction.
MIT Press, 2 edition, 2018.